

# Spring Boot

---

## 1. Spring Boot 權限檢查

---

- 集成 Spring Security 來處理用戶認證和授權
  - 設置路由和角色權限來限制資源存取
  - 使用 `@PreAuthorize` 或 `@Secured` 注解來保護方法
  - 自定義過濾器進行權限檢查
- 

## Specification

---

## 2. 當 Spec 無法理解時該怎麼辦？

---

### 1. 確認大方向

- 瞭解 Spec 的目標與背景：
    - Spec 的最終目的是什麼？
    - 它的功能或影響範圍是什麼？
- 

### 2. 找出問題點

- 標記不理解的部分。
  - 將困惑轉換為具體的問題，例如：
    - 這段描述的功能是什麼？
    - 這裡的條件或限制是什麼？
- 

### 3. 查資料補充知識

- 查詢不熟悉的術語或相關背景資料。
  - 若技術相關內容有疑問，查閱文件或範例代碼。
- 

### 4. 團隊協作

- 與需求方或撰寫 Spec 的人進行溝通，詢問其具體意圖。

- 與團隊成員討論，共同釐清模糊之處。
- 

## 5. 嘗試分解與假設

- 將需求分解為小步驟進行分析。
  - 根據現有理解列出假設場景，並進行驗證。
- 

## 6. 記錄與回饋

- 整理不理解的地方，記錄並回饋給需求方。
  - 提出改進建議（例如補充範例、定義用詞或添加圖示）。
- 

## 核心重點

- 保持主動溝通與學習。
  - Spec 模糊並非個人問題，而是需要團隊共同努力改進的部分。
- 

# JSP

---

## 3. JSP 四大變數

`page`、`request`、`session`、`application`

---

- `page`：當前頁面有效
- `request`：一次請求有效
- `session`：一次會話有效
- `application`：整個應用程序有效

使用場景：

- `page`：暫存頁面局部變數
- `request`：傳遞請求數據
- `session`：儲存用戶資訊
- `application`：儲存全局配置

## 4. 學習 JSP 的步驟

---

1. 了解 JSP 的基本概念和工作原理
2. 設置開發環境（如 Tomcat）

3. 學習 JSP 語法、EL 表達式、JSTL 標籤
  4. 實作簡單的動態頁面
  5. 將 JSP 與 Spring Boot 整合，處理表單和資料顯示
  6. 學習錯誤處理並進行專案練習
- 

## 物件導向特性

---

### 5. 物件導向語言的封裝、繼承、多型

---

1. **封裝**：隱藏內部實現，只暴露必要資訊，保護資料不被隨意修改
  2. **繼承**：子類繼承父類屬性和方法，避免重複編寫代碼，支持擴展
  3. **多型**：同一方法名可對不同物件執行不同行為，增加彈性和擴展性
- 

## Java 集合框架

---

### 6. Java Collection 是什麼？

---

Java Collection 是一組框架，提供儲存和操作物件的容器。透過這些容器，開發者可以更高效地操作資料，並解決不同場景下的數據結構需求。

#### 特點

- **提供儲存和操作物件的容器**：方便管理和操作大量數據。
  - **主要接口**：`List`、`Set`、`Map`、`Queue`。
  - **支援不同數據結構和操作方法**：例如，陣列、鏈表、哈希表等。
  - **支援泛型**：提升類型安全性，避免強制轉型。
- 

### 理解 listMap 與 mapList 的不同

#### 1. listMap

- **定義**：`List<Map<K, V>>`，表示一個包含多個 Map 的列表。
- **使用場景**：當需要操作多個 Map 並且順序很重要時，使用 listMap。

#### 2. mapList

- **定義**：`Map<K, List<V>>`，表示一個鍵對應多個值的結構（每個鍵對應一個列表）。
- **使用場景**：當需要為每個鍵存儲多個值時，使用 mapList。

### 3. 關鍵區別

- `listMap` 側重於操作多個 Map，且有順序性。
- `mapList` 側重於為單一鍵存儲多個值。

---

## JSON 的使用時機

### 1. 數據交換：

- JSON 是一種輕量級的數據交換格式，適用於後端與前端、不同服務之間的數據傳遞。

### 2. 資料儲存：

- 作為輕量儲存格式，用於記錄配置或小型資料。

### 3. 資料結構的序列化與反序列化：

- Java 與其他語言之間的資料格式轉換。

### 4. 應用場景：

- RESTful API 的請求與回應格式。
- 跨平台資料交換（例如 Java 與 JavaScript 交互）。

---

## 7. Java 集合框架 - 有序集合

### List 有加入的順序？

- List 是有序集合，會保留元素的加入順序，支持通過索引存取元素。

---

## 8. TreeMap

### 特點：

1. 基於紅黑樹實現，會自動排序鍵
2. 支援自然順序或自定義比較器排序
3. 保證鍵的順序

### 問題：

1. 操作效率比 `HashMap` 低
2. 只能使用可比較的鍵，不能使用 `null` 作為鍵
3. 在不需要排序的情況下，性能較差

---

# 物件導向特性 - 封裝

---

## 9. 封裝的用意與存取限制

---

**用意：**隱藏物件內部實現，保護資料不被隨意修改，增強程式安全性。

**存取限制：**

- **private**：僅限類內部訪問
- **protected**：子類和同包可訪問
- **public**：公開訪問
- **default**：同包內可訪問

---

## 實作分頁

---

### 10. 沒有第三方套件如何實作分頁？

---

**後端 (Spring Boot)：**

- 接收頁碼和每頁數量
- 計算查詢的起始位置 ( `offset = (page - 1) * size` )
- 使用 `LIMIT` 和 `OFFSET` 查詢數據，返回總記錄數、頁碼和數據列表

**前端 (Vue 3)：**

- 計算總頁數，生成分頁按鈕
- 監聽頁碼變化並調用 API 更新數據，渲染頁面

---

## MVC 架構

---

### 11. MVC 架構簡介

---

- **Model**：管理數據、業務邏輯和與資料庫的交互
- **View**：負責用戶界面，避免處理業務邏輯
- **Controller**：接收用戶請求，調用 Model 處理業務邏輯，將結果傳遞給 View

### 12. Servlet 和 JSP 的差異

---

## Servlet

- 控制層 (Controller) ，處理業務邏輯和請求分發

## JSP

- 視圖層 (View) ，負責動態渲染頁面，顯示資料

**總結：** Servlet 處理邏輯，JSP 負責頁面顯示和渲染。

---

# 其他

---

## 13. equals 變數前後？

---

- 要先檢查是否進行 `null` 比較，再比較物件內容
- `equals` 是用來比較兩個物件的內容是否相等，而非它們是否指向同一個記憶體位置（那是 `==` 做的事）。例如，兩個 `String` 物件即使存儲在不同位置，只要內容一樣，`equals` 會返回 `true`

## 14. log 層級在哪裡設定？

---

- 後端 (Spring Boot) 的日誌設定通常在 `application.properties` 或 `logback-spring.xml` 中配置
  - 前端 (Vue 3) 使用 `console.log` ，並通過環境變數來控制開發與生產環境中的日誌顯示
- 

# SQL

---

## 15. SQL 部分熟悉嗎，JOIN 和 INNER JOIN 差異？

---

### JOIN (預設為 INNER JOIN)

SQL的JOIN類型主要有：

- INNER JOIN：兩個表中符合條件的資料才會被選出
- LEFT JOIN：會返回左表所有記錄，即使右表沒有匹配的資料
- RIGHT JOIN：會返回右表所有記錄，即使左表沒有匹配的資料
- FULL JOIN：返回所有記錄，不論是否匹配

### INNER JOIN

- 明確指定了連接方式是 `INNER JOIN` ，功能上與 `JOIN` 完全一樣。

- 會返回兩個表中符合條件的交集。

## 16. 只會基本 SQL 嗎？

---

### 1. 子查詢 (Subquery)

- 在查詢中嵌入另一個查詢，通常用來提供資料給主查詢。

### 2. CTE (Common Table Expression)

- 使用 `WITH` 子句定義一個臨時的結果集，提升可讀性並支援遞迴查詢。

### 3. 視圖 (VIEW)

- 將查詢結果儲存為一個虛擬表，方便重複使用。

---

## 前端技術

---

## 17. HTML 和 JSP 差異？

---

- **HTML** 是靜態網頁，要修改資料需要重新渲染。
- **JSP** 是動態網頁，可以根據不同條件顯示不同內容，且能連接資料庫即時更新資料，並記住使用者的操作狀態。

## 18. EL 和 JSTL 差別？

---

- **EL (Expression Language)**：讓 JSP 頁面能夠方便地存取 `JavaBean`、`Map`、`List`、`Request`、`Session`、`Application` 等作用域中的資料，而不需要寫出繁瑣的 Java 程式碼。
- **JSTL (JavaServer Pages Standard Tag Library)**：提供一組標準的標籤，來完成邏輯處理、迴圈、國際化、SQL 操作等功能，讓 JSP 頁面邏輯變得更簡潔。

## 19. JSTL 和 `<% %>` 寫 Code 有什麼差別？

---

- **JSTL**：使用標籤語法，如 `<c:if>`、`<c:forEach>` 等，語法清晰、易讀，適合非程式設計人員與設計師閱讀和維護。
- `<% %>` (**Scriptlet**)：使用 Java 程式碼嵌入 JSP 頁面，程式碼較難閱讀，尤其邏輯較複雜時，容易導致程式碼混亂，破壞 MVC 架構的分層。

## 20. 為什麼要用 Servlet 不用 Spring RequestMapping？

---

- **Servlet**：

- 基礎且簡單
- 理解原理很重要
- 適合小型專案
- 控制比較直接
- **Spring RequestMapping :**
  - 功能強大但較複雜
  - 需要整個 Spring 框架支援
  - 適合大型專案
  - 有很多現成的功能

## 21. 連線的 close 會寫在哪裡？

---

- 在 Java 中進行資料庫操作時，連線的 `close` 方法通常會寫在 **finally 區塊** 或透過 `try-with-resources` 語法進行自動關閉，以確保無論發生任何異常，連線都能被正確關閉，釋放資源，避免連線洩漏。

## 22. AJAX（非同步）和 submit 的差別？

---

- **AJAX**：讓網頁能夠非同步地向伺服器發送請求，並獲取資料的技術，允許在不重新加載整個頁面的情況下，與伺服器進行資料交換。
- **Submit**：透過 HTML 表單的提交機制（`<form>`）將資料傳送到伺服器，提交後，瀏覽器會重新加載網頁或跳轉到指定的目標頁面。

## 23. 非同步的意思？

---

- 非同步（Asynchronous）指的是程式在執行某項任務時，不必等待該任務完成，而是繼續執行其他任務，等到該任務完成後，再去處理結果。這種方式使程式可以同時處理多個工作，不會因等待某一個任務而停滯。

## 24. 課程印象最深刻的 coding style？

---

- 駝峰式命名法，將單字連接在一起，首個單字以小寫字母開頭，後續單字的首字母大寫，便於快速識別變數或方法的名稱結構，且不依賴額外符號，簡潔明瞭。

## 25. DOM 是什麼？

---

DOM（Document Object Model）是瀏覽器用來表示網頁結構的模型。它將網頁中的每個元素（如標籤、屬性和文本）都視為物件，並且可以通過 JavaScript 動態操作這些物件。

簡單來說，DOM 是網頁的「物件導向表示」，它允許程式修改網頁的結構、樣式和內容。



## 主要概念

1. **節點 (Node)**：網頁中的每一個元素（如 `<div>`、`<p>` 等）都是 DOM 中的一個節點。
2. **層次結構 (Hierarchy)**：DOM 使用樹狀結構來表示元素之間的關係。根節點是整個文檔，其他元素則是其子節點。

## DOM 與 JavaScript

- JavaScript 可以透過 DOM 操作 HTML 元素，實現動態效果。例如，改變元素內容、修改樣式或響應使用者事件。

## 範例

```
// 透過 DOM 操作 HTML
document.getElementById("myElement").innerText = "新內容";
```

## 總結

- DOM 是網頁的程式化表示方式，讓開發者可以使用 JavaScript 操控網頁結構與內容。

---

# 交易控制 (Transaction Control)

---

## 26. 交易控制是什麼？

---

交易控制確保資料操作的一致性，管理一系列操作的執行，保證這些操作要麼全部成功，要麼全部失敗。

### 交易 (Transaction)

交易是指一組資料庫操作作為一個單位執行，保證所有操作要麼全部成功，要麼全部失敗，維持資料庫的原子性 (Atomicity) 和一致性。

### 提交 (Commit) 與回滾 (Rollback)

- **提交 (Commit)**：當交易中的所有操作成功執行後，呼叫 Commit 將變更永久寫入資料庫。
- **回滾 (Rollback)**：當交易中的某個操作失敗時，使用 Rollback 撤銷所有變更，恢復到交易開始前的狀態。

### 交易控制應用場景

交易控制應用於需要確保多個操作成功的情況（如：轉帳操作）。透過 Transaction、Commit 和 Rollback 的配合，確保資料庫操作的完整性與一致性。

